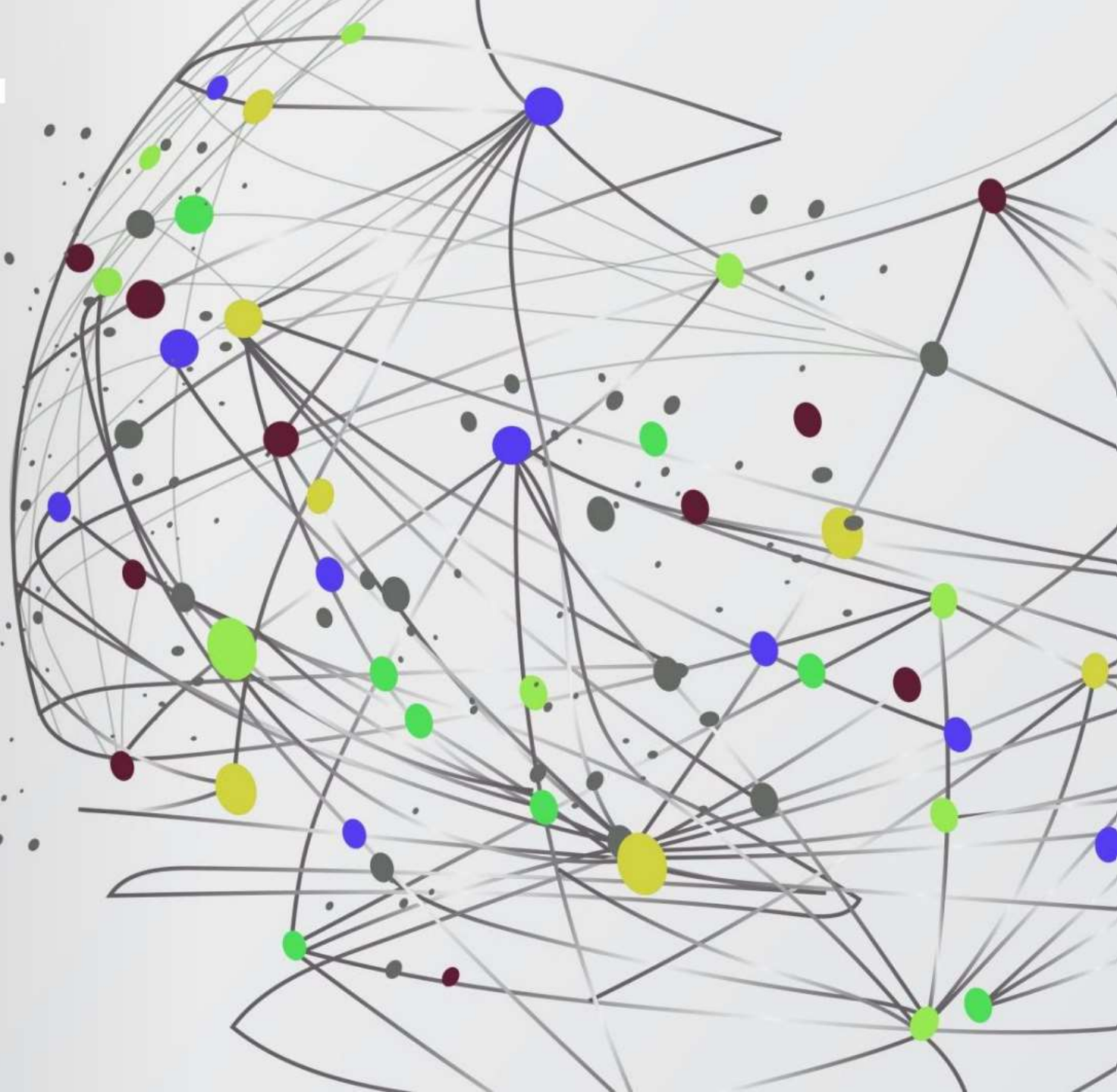

Projet 1 : Implémentation du protocole SRTP LINFO1341

Groupe 12 :

- *Benoit Marchant*
- *Maxence Felis*
- *Ramy Mansri*



Architecture globale du programme

Coté client :

- **Boucle d'écoute** : Centralise la réception, le tri via le dictionnaire `stockage_paquets` et l'envoi des ACKs.
- **Mise en cache** : Le dictionnaire permet de stocker temporairement les paquets arrivés dans le désordre pour garantir un réassemblage parfait du fichier final.
- **IPv6 et UDP** : Utilisation de datagrammes simples pour l'envoi et la réception des paquets

Côté Serveur :

- **Multi-threading** : Utilisation de `threading.Thread` pour isoler et traiter chaque requête client de manière indépendante et simultanée.
- **Files d'attente (`queue.Queue`)** : Pour assurer une communication sécurisée et synchronisée entre les threads
- **Répartiteur**: Un thread principal joue le rôle de "distributeur", dirigeant chaque message entrant vers la file d'attente du bon client pour éviter les collisions.

Champ Timestamp

- **Ce qu'on y met** : L'heure actuelle en millisecondes (`int(time.time() * 1000)`).
- **Traitement mathématique** : On utilise un modulo 2^{32} (`% 4294967296`) pour garantir que la valeur ne dépasse jamais la taille autorisée (4 octets) et éviter un crash de `struct.pack`.
- **But** :
 - **Ici dans le projet** : Ne sert qu'à respecter la structure du protocole SRTP
 - **En théorie** : Fournir les données temporelles pour calculer le RTT (latence) et adapter le RTO (timeout) dynamiquement.



Retransmission Timeout (RTO)

Valeur choisie : Nous avons choisi une valeur différente en fonction du contexte

- **2.0 secondes** pour la connexion initiale (le GET), car le serveur peut être long à répondre.
- **0.5 seconde** pour la retransmission des paquets de données en cours de transfert (boucle du Selective Repeat).

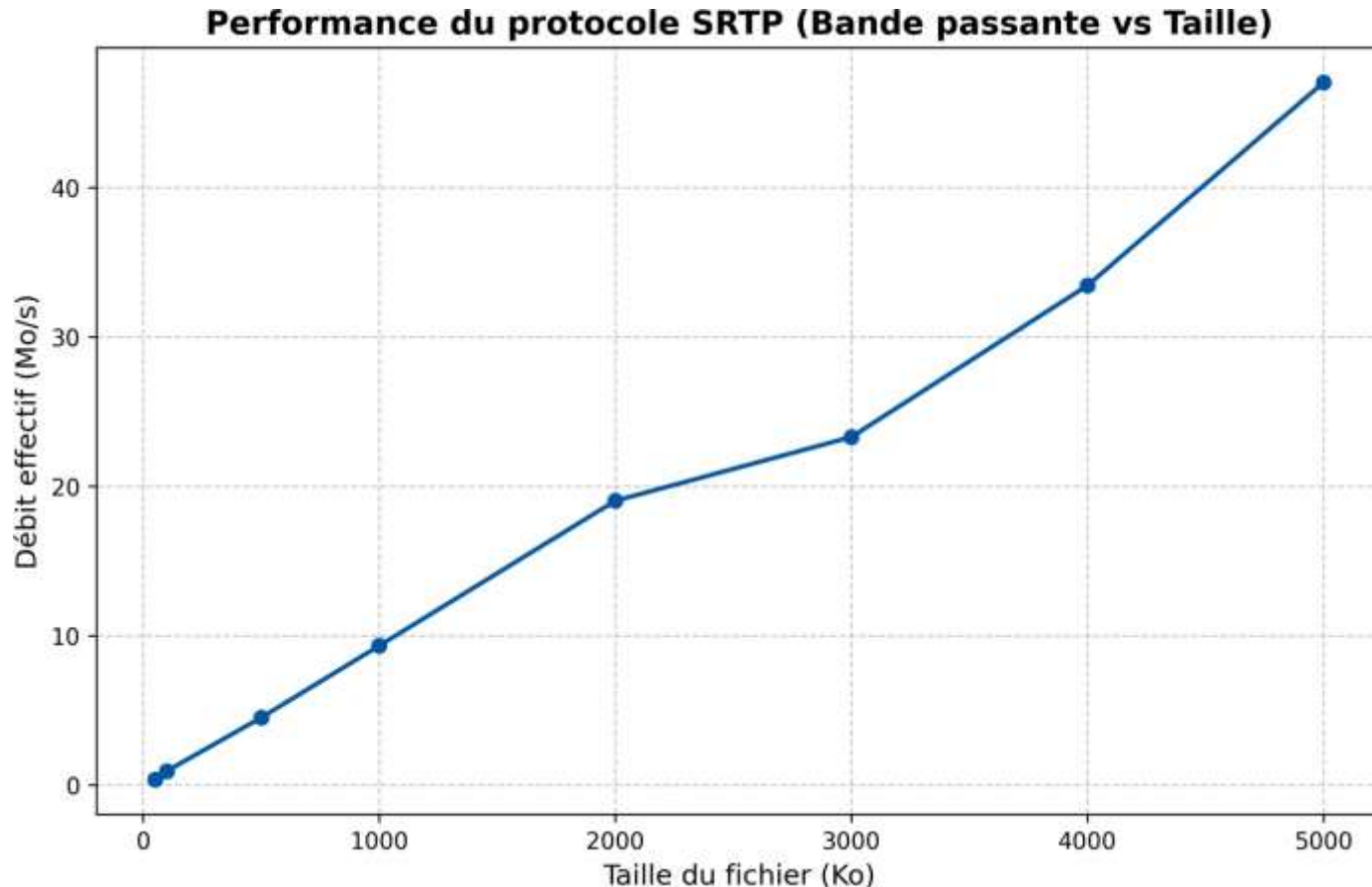
But : Garantir la fiabilité du transfert en renvoyant automatiquement tout paquet pour lequel aucune confirmation de réception (ACK) n'a été reçue avant l'expiration du délai imparti.

Limites de notre implémentation

- **Lecture du fichier complet en mémoire** : Actuellement, le serveur charge tout le fichier dans la liste blocks avant de commencer. Pour des fichiers LLM très lourds, cela peut saturer la RAM.
- **Window size** : La vitesse est bridée par le format binaire de l'en-tête défini dans packet.py
- **Taille de la fenêtre** : La taille de la fenêtre (63 maximum) limite le nombre de paquets en vol ce qui limite la vitesse théorique maximale.



Evaluation des performances



Méthodologie de test :

- Transfert de fichiers binaires allant de 50 Ko à 5 Mo.
- Tests réalisés sur un réseau sans interférence et ralentissements artificiels

Analyse des résultats :

- Sur les petits fichiers, le temps d'envoi du GET HTTP pénalise fortement la vitesse moyenne.
- Plus le fichier est volumineux, plus l'overhead est dilué.
- Conclusion : Notre implémentation est assez stable et augmente de manière +- linéaire en fonction de la taille du fichier

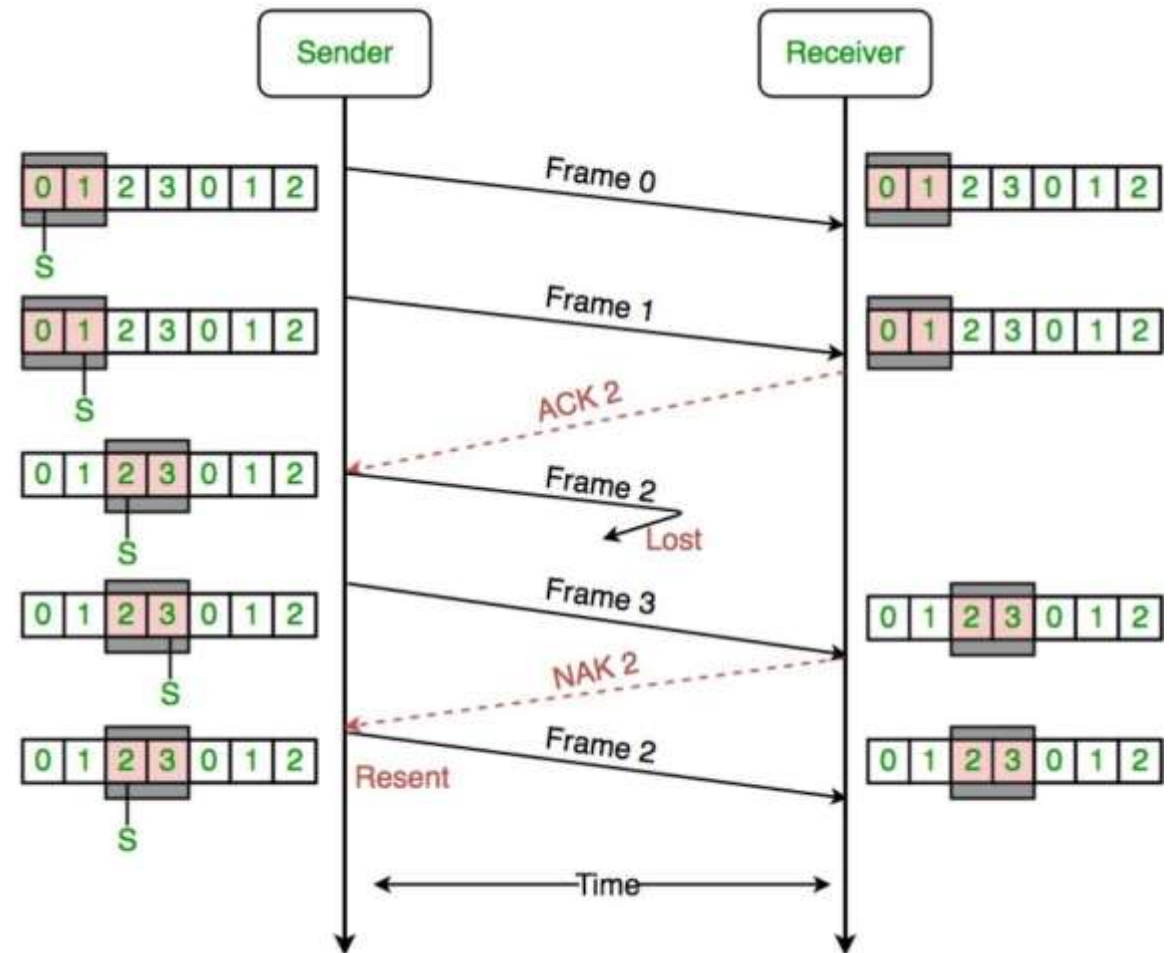
Stratégie de tests



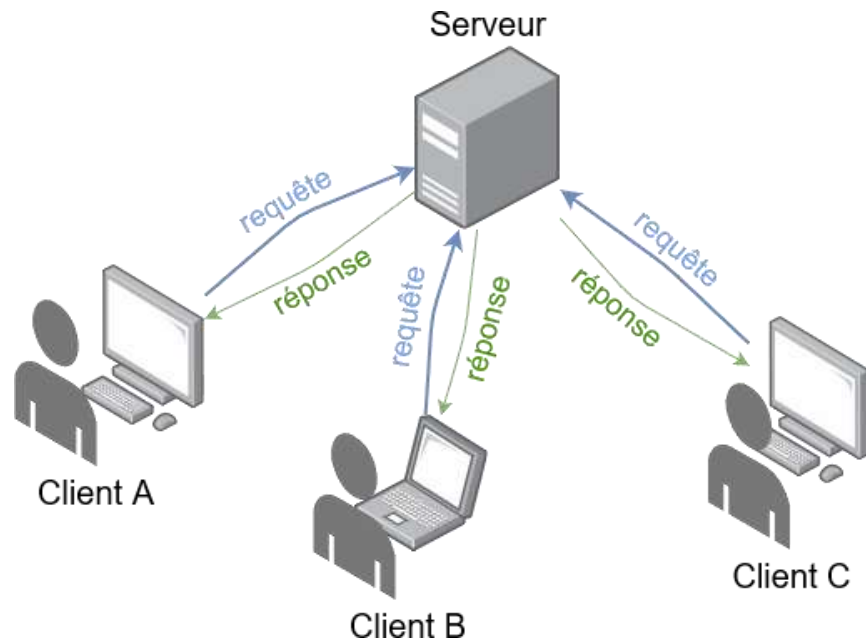
- **Vérification des paquets (test_packet.py) :**
 - Validation du formatage des en-têtes via struct.
 - Vérification du blocage des paquets corrompus par double contrôle CRC1 (en-tête) et CRC2 (données).
- **Tests Réseau (test_network.py) :**
 - **Transfert complet (5000 octets) :** Pour vérifier que le serveur découpe bien le fichier en plusieurs morceaux et que le client les recolle sans aucune erreur.
 - **Test de pertes (20%) :** Utilisation d'un faux socket pour simuler une mauvaise connexion et prouver que le Selective Repeat récupère l'intégralité du fichier.
 - **Intégrité du flux :** Validation du cycle complet (requête GET, transfert de données, ACK de fin) dans un environnement multi-threadé.
 - **Emulation avec link simulator :** Simulation d'un lien réseau pour confirmer que le fichier est transmis en intégralité sans aucune erreur ni perte d'octet

Selective Repeat

- **Mécanisme SACK (Selective ACKs) :** Le client stocke les paquets reçus hors séquence dans le dictionnaire stockage_paquets. Il confirme chaque segment individuellement ce qui permet de combler uniquement les trous dans le fichier.
- **Fenêtre Dynamique :** Le client calcule en direct sa place restante ($\text{current_window} = \text{window_size} - \text{len}(\text{stockage_paquets})$) et force le serveur à ralentir si besoin.
- **Gains de performance :** En cas de perte réseau le serveur ne retransmet que le paquet manquant. Cela évite le renvoi inutile de dizaines de paquets déjà reçus ce qui économise ainsi la bande passante.



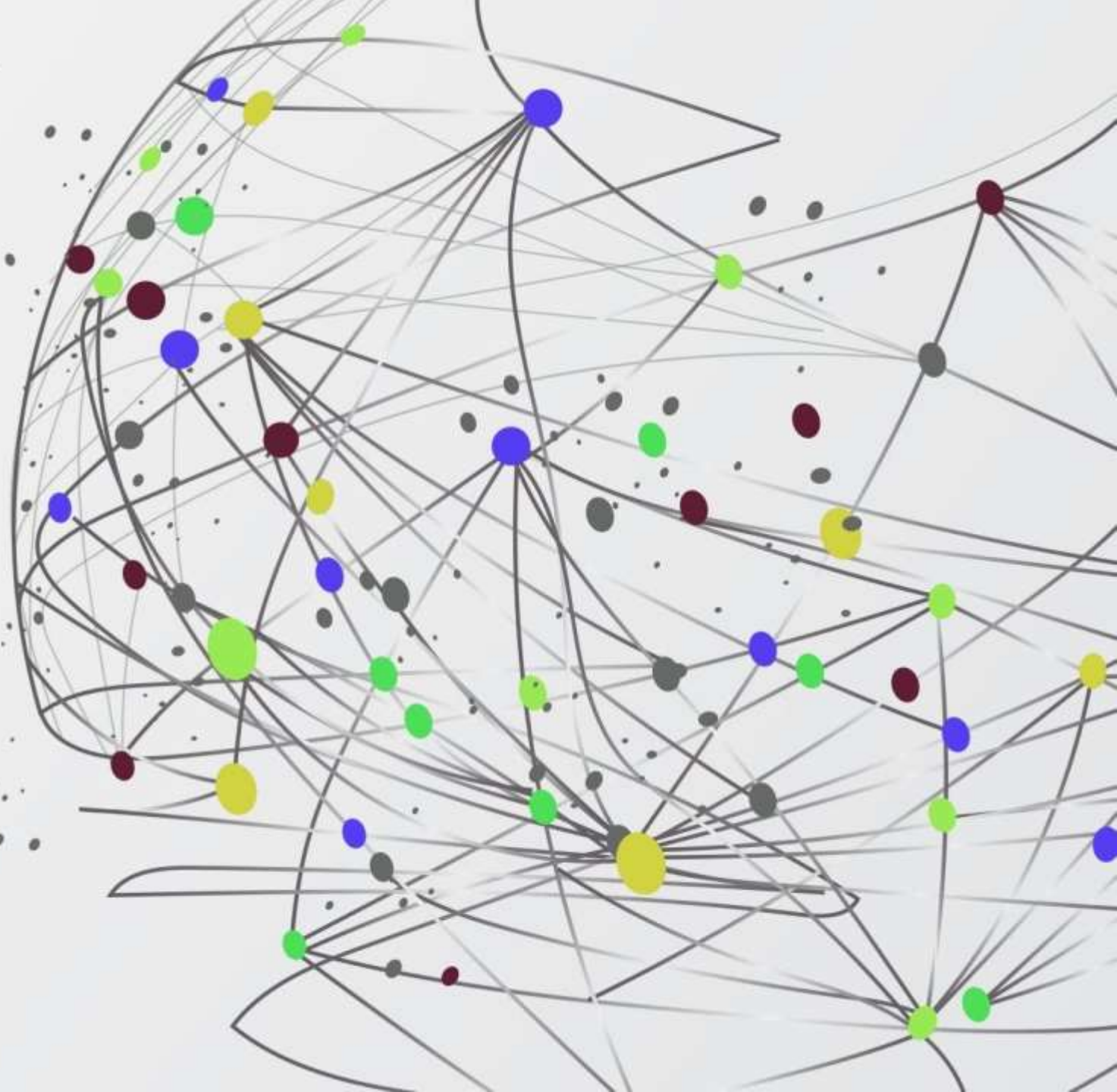
Gestion Multi-clients



- Le serveur associe chaque adresse IP/Port cliente à une queue.Queue().
- Le thread principal agit un peu comme un videur car il écoute le réseau et distribue uniquement les bons ACKs dans la file d'attente du bon thread client.

Conclusion :

- *L'objectif de transfert de données est rempli.*
- *Le code est résilient face aux pertes réseau, corruptions et déconnexions aléatoires.*



Note sur l'Utilisation de l'IA

Le règlement de l'UCL stipule que nous devons être transparent sur l'utilisation d'une IA dans le cadre d'un travail :

- Nous avons utilisé l'IA Gemini pour nous aider à comprendre/clarifier certains concepts techniques. Ainsi que pour la rédaction des commandes de lancement permettant d'exécuter simultanément le serveur et le client dans des terminaux distincts.
- Nous avons veillé à l'utiliser dans un cadre d'apprentissage et non pour faire le projet à notre place.
- L'ensemble du projet à été réfléchi et écrit par les 3 membres du groupe.